2: Kubernetes principles of operation

In this chapter, you'll learn about the major components required to build a Kubernetes cluster and deploy an app. The aim is to give you an overview of the major concepts. So don't worry if you don't understand everything straight away, we'll cover most things again as you progress through the book, and the hands-on demos will help everything fall into place.

The chapter is divided as follows:

- Kubernetes from 40K feet
- Masters and nodes
- Packaging apps for Kubernetes
- The declarative model and desired state
- Pods
- Deployments
- Services

Kubernetes from 40K feet

At the highest level, Kubernetes is two things:

- A cluster to run applications on
- An orchestrator of cloud-native microservices apps

Kubernetes as a cluster

Kubernetes is like any other cluster – a bunch of machines to host applications. We call these machines "nodes", and they can be physical servers, virtual machines, cloud instances, Raspberry Pis, and more.

A Kubernetes cluster is made of a *control plane* and *nodes*. The *control plane* exposes the API, has a scheduler for assigning work, and records the state of the cluster and apps in a persistent store. *Nodes* are where user applications run.

It can be useful to think of the *control plane* as the brains of the cluster, and the *nodes* as the muscle. In this analogy, the control plane is the brains because it implements the clever features such as scheduling, auto-scaling, and zero-downtime rolling updates. The nodes are the muscle because they do the every-day hard work of executing user applications.

Kubernetes as an orchestrator

Orchestrator is just a fancy word for a system that takes care of deploying and managing applications.

Let's look at a quick analogy.

In the real world, a football (soccer) team is made of individuals. Every individual is different, and each has a different role to play in the team – some defend, some attack, some are great at passing, some tackle, some shoot… Along comes the coach, and she or he gives everyone a position and organizes them into a team with a purpose. Things go from Figure 2.1 to Figure 2.2.

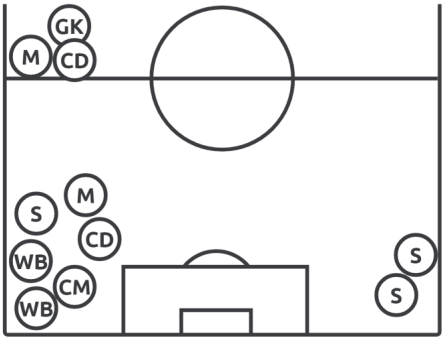

Figure 2.1

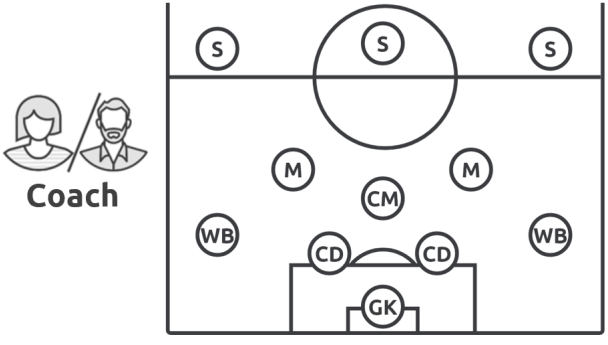

Figure 2.2

The coach also makes sure the team keeps its formation, sticks to the game-plan, and deals with any injuries and other changes in circumstances.

Well guess what, microservices apps on Kubernetes are the same.

Stick with me on this…

You start out with lots of individual specialised microservices. Some serve web pages, some do authentication, some perform searches, others persist data. Kubernetes comes along – like the coach in the football analogy – organizes everything into a useful app and keeps things running smoothly. It even responds to events and other changes in circumstance.

In the sports world this is called *coaching*. In the application world it's called *orchestration*. Kubernetes *orchestrates* cloud-native microservices applications.

How it works

You start out with an app, package it as a container, then give it to the cluster (Kubernetes). The cluster is made up of one or more *control plane nodes* and a bunch of *worker nodes*.

As already stated, control plane nodes implement the cluster intelligence. Worker nodes are where user applications run.

You follow this simple process to run applications on a Kubernetes cluster.

1. Design and write the application as small independent microservices in your favourite languages.
2. Package each microservice as its own container.
3. Wrap each container in a Kubernetes Pod.
4. Deploy Pods to the cluster via higher-level controllers such as *Deployments, DaemonSets, StatefulSets, CronJobs etc.*

Now then… this is the beginning of the book and you're not expected to know what all of this means yet. However, at a high-level, Kubernetes has several *controllers* that augment Pods with important features such as self-healing, scaling, smooth rollouts, and more. Some controllers are for stateless apps, and others are for stateful apps. You'll learn all about them as you progress through the book.

Kubernetes likes to manage applications *declaratively*. This is a pattern where you describe what you want in a set of YAML files, post them to Kubernetes, then sit back while Kubernetes makes it all happen.

But it doesn't stop there. Because the declarative pattern tells Kubernetes how an application should look, Kubernetes can watch it and make sure it doesn't stray from what you asked for. If something isn't as it should be, Kubernetes tries to fix it.

That's the big picture. Let's dig a bit deeper.

Control plane and worker nodes

As previously mentioned, a Kubernetes cluster is made of *control plane nodes* and *worker nodes*. These are Linux hosts that can be virtual machines (VM), bare metal servers in your datacenter, or instances in a private or public cloud. You can even run Kubernetes on ARM and IoT devices.

The control plane

A Kubernetes *control plane node* is a server running collection of system services that make up the control plane of the cluster. Sometimes we call them *Masters*, *Heads* or *Head nodes*.

The simplest setups run a single control plane node. However, this is only suitable for labs and test environments. For production environments, multiple control plane nodes configured for high availability (HA) is vital. Generally speaking, 3 or 5 is recommended for HA.

It's also considered a good practice not to run user applications on control plane nodes. This frees them up to concentrate entirely on managing the cluster.

Let's take a quick look at the different services making up the control plane.

The API server

The *API server* is the Grand Central of Kubernetes. **All communication, between all components, must go through the API server**. We'll get into the detail later, but it's important to understand that internal system components, as well as external user components, **all** communicate via the API server – *all roads lead to the API Server*.

It exposes a RESTful API that you `POST` YAML configuration files to over HTTPS. These YAML files, which we sometimes call *manifests*, describe the desired state of an application. This desired state includes things like which container image to use, which ports to expose, and how many Pod replicas to run.

All requests to the API server are subject to authentication and authorization checks. Once these are done, the config in the YAML file is validated, persisted to the cluster store, and work is scheduled to the cluster.

The cluster store

The cluster store is the only *stateful* part of the control plane and persistently stores the entire configuration and state of the cluster. As such, it's a vital component of every Kubernetes cluster – no cluster store, no cluster.

The cluster store is currently based on `etcd`, a popular distributed database. As it's the *single source of truth* for a cluster, you should run between 3-5 etcd replicas for high-availability, and you should provide adequate ways to recover when things go wrong. A default installation of Kubernetes installs a replica of the cluster store on every control plane node and automatically configures HA.

On the topic of *availability*, etcd prefers consistency over availability. This means it doesn't tolerate *split-brains* and will halt updates to the cluster in order to maintain consistency. However, if this happens, user applications should continue to work, you just won't be able to update the cluster config.

As with all distributed databases, consistency of writes to the database is vital. For example, multiple writes to the same value originating from different places need to be handled. etcd uses the popular RAFT consensus algorithm to accomplish this.

The controller manager and controllers

The controller manager implements all the background controllers that monitor cluster components and respond to events.

Architecturally, it's a *controller of controllers*, meaning it spawns all the independent controllers and monitors them.

Some of the controllers include the Deployment controller, the StatefulSet controller, and the ReplicaSet controller. Each one is responsible for a small subset of cluster intelligence and runs as a background watch-loop constantly watching the API Server for changes.

The aim of the game is to ensure the *observed state* of the cluster matches the *desired state* (more on this shortly).

The logic implemented by each controller is as follows, and is at the heart of Kubernetes and declarative design patterns.

1. Obtain desired state
2. Observe current state
3. Determine differences

4. Reconcile differences

Each controller is also extremely specialized and only interested in its own little corner of the Kubernetes cluster. No attempt is made to over-complicate design by implementing awareness of other parts of the system – each controller takes care of its own business and leaves everything else alone. This is key to the distributed design of Kubernetes and adheres to the Unix philosophy of building complex systems from small specialized parts.

> **Terminology**: Throughout the book we'll use terms like *controller*, *control loop, watch loop,* and *reconciliation loop* to mean the same thing.

The scheduler

At a high level, the scheduler watches the API server for new work tasks and assigns them to appropriate healthy worker nodes. Behind the scenes, it implements complex logic that filters out nodes incapable of running tasks, and then ranks the nodes that are capable. The ranking system is complex, but the node with the highest ranking score is selected to run the task.

When identifying nodes capable of running a task, the scheduler performs various predicate checks. These include is the node tainted, are there any affinity or anti-affinity rules, is the required network port available on the node, does it have sufficient available resources etc. Any node incapable of running the task is ignored, and those remaining are ranked according to things such as does it already have the required image, how much free resource does it have, how many tasks is it currently running. Each is worth points, and the node with the most points is selected to run the task.

If the scheduler doesn't find a suitable node, the task isn't scheduled and gets marked as pending.

The scheduler isn't responsible for *running* tasks, just picking the nodes to run them. A *task* is normally a Pod/container. You'll learn about Pods and containers in later chapters.

The cloud controller manager

If you're running your cluster on a supported public cloud platform, such as AWS, Azure, GCP, or Linode, your control plane will be running a *cloud controller manager*. Its job is to facilitate integrations with cloud services, such as instances, load-balancers, and storage. For example, if your application asks for an internet-facing load-balancer, the cloud controller manager provisions a load-balancer from your cloud and connects it to your app.

Control Plane summary

Kubernetes control plane nodes are servers that run the cluster's control plane services. These services are the brains of the cluster where all the control and scheduling decisions happen. Behind the scenes, these services include the API server, the cluster store, scheduler, and specialised controllers.

The API server is the front-end into the control plane and all instructions and communication pass through it. By default, it exposes a RESTful endpoint on port 443.

Figure 2.3 shows a high-level view of a Kubernetes control plane node.

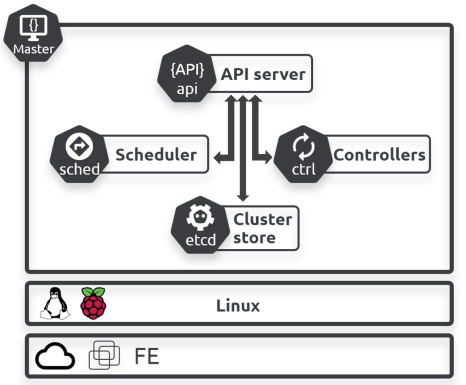

Figure 2.3 - control plane node

Worker nodes

Nodes are servers that are the workers of a Kubernetes cluster.

At a high-level they do three things:

1. Watch the API server for new work assignments
2. Execute work assignments
3. Report back to the control plane (via the API server)

As you can see in Figure 2.4, they're a bit simpler than *control plane nodes*.

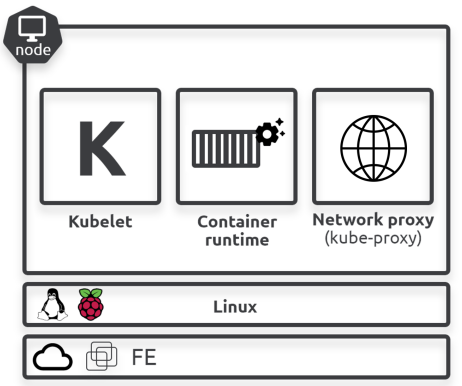

Figure 2.4 - Kubernetes node

Let's look at the three major components of a node.

Kubelet

The kubelet is main Kubernetes agent and runs on every cluster node. In fact, it's common to use the terms *node* and *kubelet* interchangeably.

When you join a node to a cluster, the process installs the kubelet, which is then responsible for registering it with the cluster. This process registers the node's CPU, memory, and storage into the wider cluster pool.

One of the main jobs of the kubelet is to watch the API server for new work tasks. Any time it sees one, it executes the task and maintains a reporting channel back to the control plane.

If a kubelet can't run a task, it reports back to the control plane and lets the control plane decide what actions to take. For example, if a kubelet cannot execute a task, it is **not** responsible for finding another node to run it on. It simply reports back to the control plane and the control plane decides what to do.

Container runtime

The kubelet needs a *container runtime* to perform container-related tasks – things like pulling images and starting and stopping containers.

In the early days, Kubernetes had native support for Docker. More recently, it's moved to a plugin model called the Container Runtime Interface (CRI). At a high-level, the CRI masks the internal machinery of Kubernetes and exposes a clean documented interface for 3rd-party container runtimes to plug into.

Kubernetes is dropping support for Docker as a container runtime. This is because Docker is bloated and doesn't support the CRI (requires a shim). containerd is replacing it as the most common container runtime on Kubernetes.

> **Note:** `containerd` (pronounced "container-dee") is the container supervisor and runtime logic stripped out from the Docker Engine. It was donated to the CNCF by Docker, Inc. and has a lot of community support. Other CRI container runtimes exist.

Kube-proxy

The last piece of the *node* puzzle is the kube-proxy. This runs on every node and is responsible for local cluster networking. It ensures each node gets its own unique IP address, and it implements local iptables or IPVS rules to handle routing and load-balancing of traffic on the Pod network. More on all of this later in the book.

Kubernetes DNS

As well as the various control plane and node components, every Kubernetes cluster has an internal DNS service that is vital to service discovery.

The cluster's DNS service has a static IP address that is hard-coded into every Pod on the cluster. This ensures every container and Pod can locate it and use it for discovery. Service registration is also automatic. This means apps don't need to be coded with the intelligence to register with Kubernetes service discovery.

Cluster DNS is based on the open-source CoreDNS project (https://coredns.io/).

Now that you understand the fundamentals of control plane nodes and worker nodes, let's switch gears and see how to package applications to run on Kubernetes.

Packaging apps for Kubernetes

An application needs to tick a few boxes to run on a Kubernetes cluster. These include.

1. Packaged as a container
2. Wrapped in a Pod
3. Deployed via a declarative manifest file

It goes like this…

You write an application microservice in a language of your choice. Then you build it into a container image and store it in a registry. At this point, the application service is *containerized*.

Next, you define a Kubernetes Pod to run the containerized application. At the kind of high level we're at, a Pod is just a wrapper that allows a container to run on a Kubernetes cluster. Once you've defined the Pod, you're ready to deploy the app to Kubernetes.

While it's possible to run static Pods like this on a Kubernetes cluster, the preferred model is to deploy all Pods via higher-level controllers. The most common controller is the *Deployment*. It offers scalability, self-healing, and rolling updates for stateless apps. You define *Deployments* in YAML manifest files that specify things how many replicas to deploy and how to perform updates.

Figure 2.5 shows application code packaged as a *container*, running inside a *Pod*, managed by a *Deployment* controller.

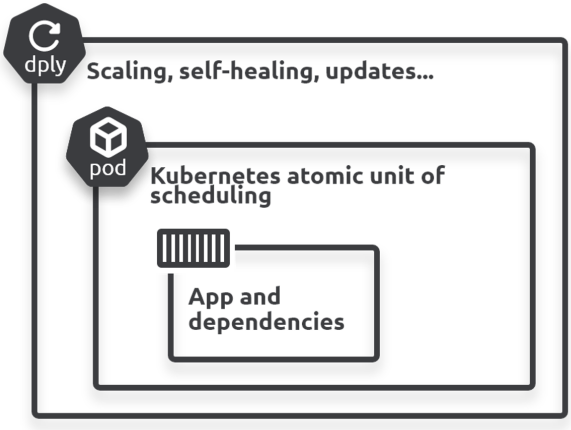

Figure 2.5

Once everything is defined in the *Deployment* YAML file, you can use the Kubernetes command-line tool to post it to the API server as the *desired state* of the application, and Kubernetes will implement it.

Speaking of desired state…

The declarative model and desired state

The *declarative model* and the concept of *desired state* are at the very heart of Kubernetes. So, it's vital you understand them.

In Kubernetes, the declarative model works like this.

1. Declare the desired state of an application microservice in a manifest file
2. Post it to the API server
3. Kubernetes stores it in the cluster store as the application's *desired state*
4. Kubernetes implements the desired state on the cluster
5. A controller makes sure the *observed state* of the application doesn't vary from the *desired state*

Let's look at each step in a bit more detail.

Manifest files are written in simple YAML and tell Kubernetes what an application should look like. This is called *desired state*. It includes things such as which image to use, how many replicas to run, which network ports to listen on, and how to perform updates.

Once you've created the manifest, you post it to the API server. The easiest way to do this is with the `kubectl` command-line utility. This sends the manifest to the control plane as an HTTP POST, usually on port `443`.

Once the request is authenticated and authorized, Kubernetes inspects the manifest, identifies which controller to send it to (e.g. the *Deployments controller*), and records the config in the cluster store as part of overall *desired state*. Once this is done, any required work tasks get scheduled to cluster nodes where the kubelet co-ordinates the hard work of pulling images, starting containers, attaching to networks, and starting application processes.

Finally, controllers run as background reconciliation loops that constantly monitor the state of things. If the *observed state* varies from *desired state*, Kubernetes performs the tasks are necessary to reconcile the issue.

It's important to understand that what we've described is the opposite of the traditional *imperative model*. The imperative model is where you write long scripts of platform-specific commands to build and monitor things.

Not only is the declarative model a lot simpler than long scripts with lots of imperative commands, it also enables self-healing, scaling, and lends itself to version control and self-documentation. It does all of this by telling the cluster *how things should look*. If they start to look different, the appropriate controller notices the discrepancy and does all the hard work to reconcile the situation.

> **Terminology**: *observed state, actual state* and *current state* all mean the same thing.

Let's consider an example.

Declarative example

Assume you have an app with a desired state that includes `10` replicas of a web front-end Pod. If a node running two replicas fails, the *observed state* will be reduced to `8` replicas, but *desired state* will still be `10`. This will be observed by a controller and Kubernetes will schedule two new replicas to bring the total back up to `10`.

The same thing will happen if you intentionally scale the desired number of replicas up or down. You could even change the image you want to use (this is called a rollout). For example, if the app is currently using `v2.00` of an

image, and you update the desired state to specify `v2.01`, the relevant controller will notice the difference and go through the process of updating the cluster so all 10 replicas are running the new version.

To be clear. Instead of writing a complex script to step through the entire process of updating every replica to the new version, you simply tell Kubernetes you want the new version, and Kubernetes does the hard work for you.

Despite how simple this might seem, it's extremely powerful and at the very heart of how Kubernetes operates.

Pods

In the VMware world, the atomic unit of scheduling is the virtual machine (VM). In the Docker world, it's the container. Well… in the Kubernetes world, it's the *Pod*.

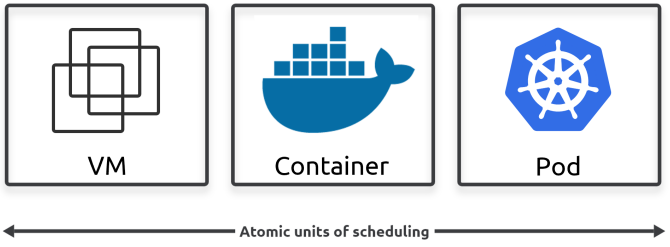

Figure 2.6

It's true that Kubernetes runs containerized apps. However, Kubernetes demands that every container runs inside a Pod.

> **Note:** Pods are objects in the Kubernetes API, so we capitalize the first letter. This might annoy you if you're passionate about language and proper use of capitalization. However, it adds clarity and the official Kubernetes docs are moving towards this standard.

Pods and containers

The very first thing to understand is that the term *Pod* comes from a *pod of whales* – in the English language we call a group of whales a *pod of whales*. As the Docker logo is a whale, Kubernetes ran with the whale concept and that's why we have *Pods*.

The simplest model is to run a single container in every Pod. This is why we often use the terms "Pod" and "container" interchangeably. However, there are advanced use-cases that run multiple containers in a single Pod. Powerful examples of multi-container Pods include:

- Service meshes
- Web containers supported by a *helper* container pulling updated content
- Containers with a tightly coupled log scraper

The point is, a Kubernetes Pod is a construct for running one or more containers. Figure 2.7 shows a multi-container Pod.

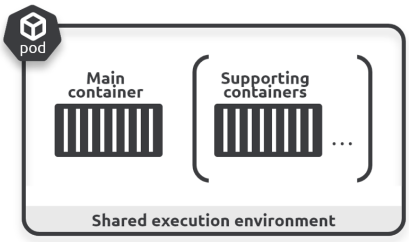

Figure 2.7

Pod anatomy

At the highest-level, a *Pod* is a ring-fenced environment to run containers. Pods themselves don't actually run applications – applications always run in containers, the Pod is just a sandbox to run one or more containers. Keeping it high level, Pods ring-fence an area of the host OS, build a network stack, create a bunch of kernel namespaces, and run one or more containers.

If you're running multiple containers in a Pod, they all share the same Pod environment. This includes the network stack, volumes, IPC namespace, shared memory, and more. As an example, this means all containers in the same Pod will share the same IP address (the Pod's IP). This is shown in Figure 2.8.

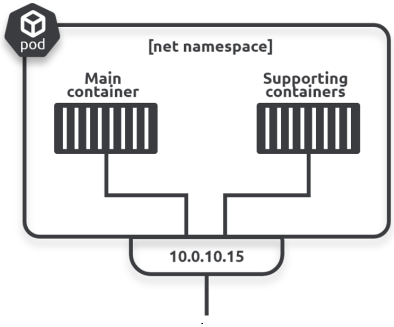

Figure 2.8

If two containers in the same Pod need to talk to each other (container-to-container within the Pod) they can use the Pod's `localhost` interface as shown in Figure 2.9.

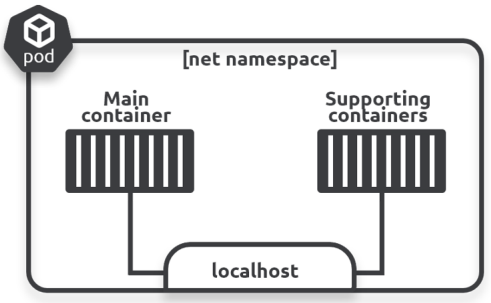

Figure 2.9

Multi-container Pods are ideal when you have requirements for tightly coupled containers that may need to share memory and storage. However, if you don't *need* to tightly couple containers, you should put them in their own Pods and loosely couple them over the network. This keeps things clean by having each Pod dedicated to a single task. However, it creates a lot of potentially un-encrypted network traffic. You should seriously consider using a service mesh to secure traffic between Pods and application services.

Pods as the unit of scaling

Pods are also the minimum unit of scheduling in Kubernetes. If you need to scale an app, you add or remove Pods. You **do not** scale by adding more containers to existing Pods. Multi-container Pods are only for situations where two different, but complimentary, containers need to share resources. Figure 2.10 shows how to scale the `nginx` front-end of an app using Pods as the unit of scaling.

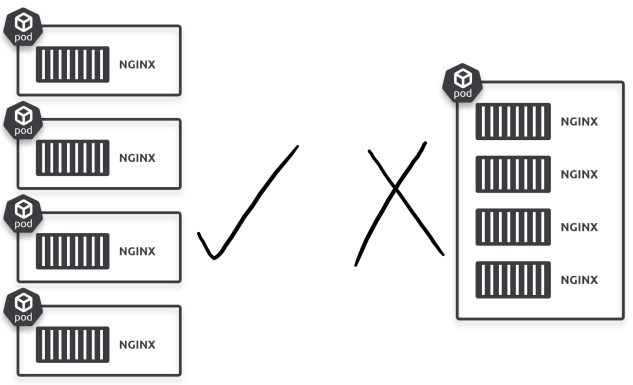

Figure 2.10 - Scaling with Pods

Pods - atomic operations

The deployment of a Pod is an atomic operation. This means a Pod is only ready for service when all its containers are up and running. The entire Pod either comes up and is put into service, or it doesn't, and it fails.

A single Pod can only be scheduled to a single node - you cannot schedule a single Pod across multiple nodes. This is also true of multi-container Pods – all containers in the same Pod run on the same node.

Pod lifecycle

Pods are mortal. They're created, they live, and they die. If they die unexpectedly, you don't bring them back to life. Instead, Kubernetes starts a new one in its place. However, even though the new Pod looks, smells, and feels like the old one, it isn't. It's a shiny new Pod with a shiny new ID and IP address.

This has implications on how you design your applications. Don't design them to be tightly coupled to a particular instance of a Pod. Instead, design them so that when Pods fail, a totally new one (with a new ID and IP address) can pop up somewhere else in the cluster and seamlessly take its place.

Pod immutability

Pods are also immutable – this means you don't change them once they're running.

Once a Pod is running, you never change its configuration. If you need to change or update it, you replace it with a new one running the new configuration. When we've talked about *updating Pods*, we've really meant delete the old one and replace it with a new one.

Deployments

Most of the time you'll deploy Pods indirectly via higher-level controllers. Examples of higher-level controllers include *Deployments*, *DaemonSets*, and *StatefulSets*.

As an example, a Deployment is a higher-level Kubernetes object that wraps around a Pod and adds features such as self-healing, scaling, zero-downtime rollouts, and versioned rollbacks.

Behind the scenes, Deployments, DaemonSets and StatefulSets are implemented as controllers that run as watch loops constantly observing the cluster making sure observed state matches desired state.

Service objects and stable networking

You've just learned that Pods are mortal and can die. However, if they're managed via higher level controllers, they get replaced when they fail. But replacements come with totally different IP addresses. This also happens with rollouts and scaling operations. Rollouts replace old Pods with new ones with new IPs. Scaling up adds new Pods with new IP addresses, whereas scaling down takes existing Pods away. Events like these cause a lot of *IP churn*.

The point we're making is that **Pods are unreliable**, and this poses challenges…

Assume you've got a microservices app with a bunch of Pods performing video rendering. How will this work if other parts of the app that use the rendering service can't rely on rendering Pods being there when needed?

This is where *Services* come in to play. They provide reliable networking for a set of Pods.

Figure 2.11 shows the uploader microservice talking to the renderer microservice via a Kubernetes Service object. The Service (capital "S" because it's a Kubernetes API object) is providing a reliable name and IP. It's also load-balancing requests to the two renderer Pods behind it.

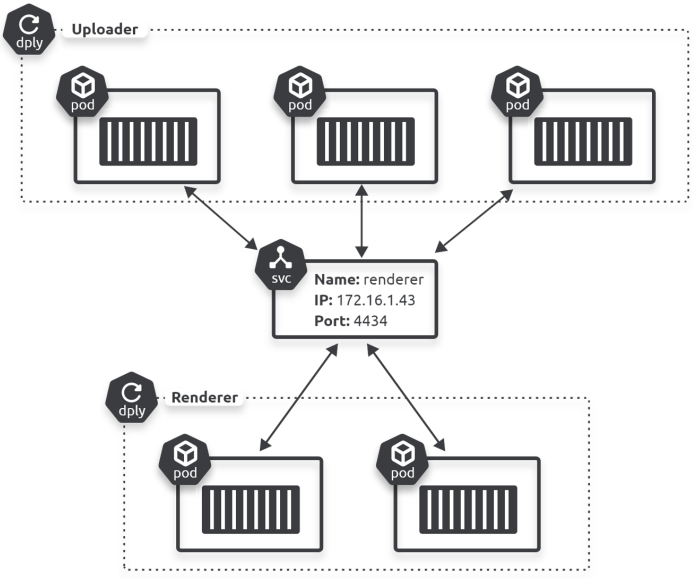

Figure 2.11

Digging into a bit more detail. Services are fully-fledged objects in the Kubernetes API – just like Pods and Deployments. They have a front-end consisting of a stable DNS name, IP address, and port. On the back-end, they load-balance traffic across a dynamic set of Pods. As Pods come and go, the Service observes this, automatically updates itself, and continues to provide that stable networking endpoint.

The same applies if you scale the number of Pods up or down. New Pods are seamlessly added to the Service and will receive traffic. Terminated Pods are seamlessly removed from the Service and will not receive traffic.

That's the job of a Service – it's a stable network abstraction point that provides TCP and UDP load-balancing across a dynamic set of Pods.

As they operate at the TCP and UDP layer, they don't possess *application intelligence*. This means they cannot provide application-layer host and path routing. For that, you need an *Ingress*, which understands HTTP and provides host and path-based routing.

That's the basics. Services bring stable IP addresses and DNS names to the unstable world of Pods.

Chapter summary

In this chapter, we introduced some of the major components of a Kubernetes cluster.

Control plane nodes are servers where the control plane components run. They can be physicals, VMs, cloud instances and even Raspberry Pis.

Under-the-hood, the control-plane comprises several services, including the API server that exposes a public REST interface to the cluster and where all the scheduling decisions are made. Multi-master HA is important for production environments.

Nodes are servers where user applications run. They can also be physicals, virts, cloud instances and Raspberry Pis.

Every node runs a service called the kubelet that registers it with the cluster and communicates with the API server. It watches the API server for new work tasks and maintains a reporting channel. They also have a container runtime and the kube-proxy service. The container runtime is responsible for all container-related operations. The kube-proxy is responsible for networking on the node.

We also talked about some of the major Kubernetes API objects such as Pods, Deployments, and Services. The Pod is the basic building-block that application containers run in. Deployments add self-healing, scaling and updates. Services add stable networking and basic load-balancing.

Now that we've covered the basics, let's get into the detail.